Contents

1 Strings, lists, tuples, and dictionaries **2**

2 Length (`len`) function **2**

3 Indexing **2**

3.1 Slicing . . . 3

3.2 Indexing and slicing lists and tuples . . . 3

3.3 Modifying lists . . . 3

4 Arithmetic operators **4**

5 Membership operator (`in`) **4**

6 Methods **4**

6.1 The None data type (NoneType) . . . 5

6.2 List methods . . . 5

7 List comprehension **6**

8 Dictionaries (`dict`) **6**

8.1 Length, indexing, and modifying a dictionary . . . 6

8.2 Membership operator (`in`) . . . 7

8.3 Dictionary methods . . . 7

9 Iteration **7**

1 Strings, lists, tuples, and dictionaries

This week, we revisit strings and introduce several new data types. We start with the definition of strings, lists, and tuples, since there are several similarities between them.

```
eg_string = "Hello, World!"
eg_list = [1, 3.4, "this is a list", True, False, True]
eg_tuple = (1, 3.4, "this is a tuple", True)
```

Strings are sequences of characters enclosed by matching quotation marks (single or double). We have also seen some other data types can also be converted into strings using the `str` function.

Lists are sequentially ordered collections of objects of any data type. They can be defined by enclosing several objects in Python in square brackets (`[]`) and separating them with commas.

Tuples, like lists, are sequentially ordered collections of objects of any data type. Instead of square brackets that are used in lists, tuples use parentheses (`()`). Tuples differ from lists in that **tuples cannot be modified**, unlike lists. We will see this behaviour later.

2 Length (`len`) function

These data types are a collection or sequence of data, so a natural question is: "how many objects are in this collection?" The `len` function works on all of these data types:

```
print(len(eg_string))     # 16, since there are 16 characters
print(len(eg_list))       # 6, since there are 6 objects
print(len(eg_tuple))      # 4, since there are 4 objects
```

3 Indexing

Indexing and slicing allow us to select *parts* of these collections. We will focus on indexing and slicing strings first. The characters in a string have an order to them. For example, given the string `"Hello, World!"`, we can say what the first, second, last, etc. characters are. We can access each character using an *index*, which is indicative of its position.

Each position in a string can be referenced two ways, with a non-negative index, and with a negative index. Python is zero-indexed, so the first element has index `0`, and increases forwards. The last element has an index of `-1`, and decreases backwards.

letter	H	e	l	l	o	,		W	o	r	l	d	!
Positive index	0	1	2	3	4	5	6	7	8	9	10	11	12
Negative index	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Indexing allows us to get a *single character* from a string, and in Python is done with *square bracket notation*. It is the same square brackets that are used to construct lists - but when used for indexing, the square brackets follow an object and are not standalone.

```
eg_string = "Hello, World!"
print(eg_string[0])     # character at index 0, 'H'
print(eg_string[4])     # character at index 4, 'o'
print(eg_string[-1])    # character at index -1, '!'
```

3.1 Slicing

Slicing gets *multiple characters* from a string, and uses the syntax `my_string[start:end:step]`. Like the `range()` function, you can pass a start, end, and step, and the slice will always be exclusive of the end. `start` takes the index `0` if unspecified, `end` takes the index **`len`**`(my_string)` if unspecified, and `step` takes a default value of `1` if unspecified.

You can mix both positive indices and negative indices, and should think of it as getting characters between two positions. This is rather uncommon, though.

```
print(eg_string[0:3])       # characters between index 0 and 3, "Hel"
print(eg_string[:6])        # characters until index 6, "Hello,"
print(eg_string[-6:])       # characters from index -6, "World!"
print(eg_string[-10:10])    # "lo, Wor"
print(eg_string[::-1])      # "!dlroW ,olleH"
```

3.2 Indexing and slicing lists and tuples

Indexing and slicing lists and tuples is exactly the same as for strings. The only difference is that each item in a list or tuple is not necessarily just a single character, as in a string.

```
eg_list = [1, 3.4, "this is a list", True, False, True]
eg_tuple = (1, 3.4, "this is a tuple", True)
print(eg_list[-4])          # "this is a list"
print(eg_list[3:5])         # [True, False]
print(eg_tuple[::-1])       # (True, "this is a tuple", 3.4, 1)
```

3.3 Modifying lists

This indexing and slicing operation allows for the contents of lists to be modified.

```
number_list = [1, 3, 5, 5, 9]
print(number_list)              # [1, 3, 5, 5, 9]
number_list[3] = 7              # modify value at index 3
print(number_list)              # [1, 3, 5, 7, 9]
number_list[0:3] = (0, 2, 4)    # modify values at indices 0, 1, 2
print(number_list)              # [0, 2, 4, 7, 9]
```

Lists are *mutable* (can mutate and change). Strings, and tuples, however, cannot be modified, and are *immutable*.

```
eg_string = "Hello, World!"
# eg_string[0] = "h"            # causes an error
eg_tuple = (1, 3.4, "this is a tuple", True)
# eg_tuple[0] = 1.1             # causes an error
```

The only way to "change" strings and tuples is to create new ones, potentially using parts of other strings or tuples.

```
new_string = "h" + eg_string[1:]
print(new_string)

# Single element tuples have a comma!
new_tuple = (1.1, ) + eg_tuple[1:]
print(new_tuple)
```

4 Arithmetic operators

As in the lab 3 exercises where you experimented with operations on strings, you can concatenate (add) two strings together with the + operator, and also repeat a string with the * operator. The same behaviour occurs with lists and tuples.

```
print("a" * 2 + "bc" + "123")       # aabc123
print(["a", "b"] * 2 + [2, 3])      # ["a", "b", "a", "b", 2, 3]
print(("c", "d") * 2 + (4, 5))      # ("c", "d", "c", "d", 4, 5)
```

5 Membership operator (`in`)

The `in` operator tests whether the LHS object is in the RHS collection. For strings, the LHS must be a *contiguous substring* in the RHS string. For lists, the LHS must be a *complete match* of an element in the RHS list.

```
print(" " in "Hello, World!")       # True
print("ello" in "Hello, World!")    # True
print("abc" in "aabbcc")            # False, 'a', 'b', 'c' all occur, but
    not consecutively
print("bcc" in "aabbcc")            # True, 'bcc' occurs as a substring

print(1 in [1, 2, 3])               # True
print("abc" in ['a', 'b', 'c'])     # False
print("a" in ['abc'])               # False
print(["abc"] in ["abc"])           # False: the RHS contains the string "
    abc", not a list!
# The same for tuples as for lists.
```

6 Methods

Methods are functions that are attached to and typically act on a specific data type. For example, `str.upper()` is a string method that acts on strings that converts a string to uppercase. The dot notation indicates that the `upper()` function comes from `str`.

```
print(str.upper("abc"))
```

Another equivalent, and more common way of uppercasing "abc" is shown below, where the dot notation indicates that `upper()` acts on the object before the dot, i.e. the string `"abc"`. Behind the

scenes, the below code also calls `str.upper()`, but passes the object before the dot as the first argument. The same behaviour holds for any other method.

```
print("abc".upper())        # call str.upper() with "abc" as the subject
                            # i.e. calls str.upper("abc")
```

To see what methods are defined on an object, you can use the `help()` function on a type, e.g. `help(str)`, `help(list)`, or any other type. There are *many*, and it is not expected for anyone to remember them all. In fact, you should be able to implement some of these methods yourself.

Here are some examples of string methods, which you are also asked to implement in the exercises:

```
print("abcba".find('b'))          # 1 - the index of the first occurrence
    of 'b'
print("  a b c  ".strip())        # 'a b c' - with trailing spaces removed
print("abcbcba".split('b'))       # ['a', 'c', 'c', 'a']
print('!'.join(['a', 'b', 'c']))  # 'a!b!c'
```

6.1 The None data type (NoneType)

Before moving on to list methods, there is one last special data type to take note of, which is called `NoneType`. While integers, floats, and strings can take many possible values (e.g. `1`, `2`, `2.718`, `'abc'`, etc.), the `NoneType` data type has only one value, which we also call `None`. It is meant to indicate 'nothing'.

While this `NoneType` seems very trivial and useless, it is important to be aware of it, because there are functions which return a `None` value - for example, the `print()` function. There are other examples below, including the list methods `sort()` and `reverse()`.

```
print(None)
print(type(None))
x = print("abcdef")     # we print "abcdef"
print(x)                # x now has value None
```

6.2 List methods

Lists have their own methods, which act and modify the list, the most important of which are `list.append()` and `list.pop()`. You are also asked to implement some of them in the exercises.

```
l = [0, 1, 2, 3]
l.append(4)           # Add element 4 to the end of the list.
l.extend([5, 6, 7])   # Adds all the elements passed to the list.
x = l.pop(0)          # Removes and returns the element at index 0.
l.insert(0, 9)        # Inserts at index 0 the element 9.
l.remove(5)           # Remove the first occurrence of 5, if found.
l.sort()              # Sorts the list in-place. Returns nothing.
l.reverse()           # Reverses the list in-place. Returns nothing.
```

Some methods do not return anything. It is a common mistake to do the following:

```
l = [3, 2, 1]      # create a list
l = l.sort()       # l.sort() sorts in-place, and returns None.
print(l)           # l is None
l = l.reverse()    # causes an error
```

For now, just remember that assigning a variable `list.sort()` and `list.reverse()` like above will cause that variable to be `None`. We will study this behaviour next week, when we learn to define our own functions.

7 List comprehension

A list comprehension is a concise way to create a new list from an existing list or other iterable. It consists of:

1. an expression: The operation you want to apply to each element of the input iterable.
2. an iterable: The list or other iterable that you want to process.
3. a condition (optional): A condition that filters out elements from the input iterable.

```
# [expression for i in iterable if condition]

# apply f(n) = 2*n for every n in [0, 1, 2, 3, 4]
l = [2*n for n in range(5)]           # [0, 2, 4, 6, 8]

# apply f(c) = c + "1" for every c in "ABC"
l = [c + "1" for c in "ABC"]           # ["A1", "B1", "C1"]

# apply f(x, y) = (x, y) for every x in [1, 2], y in ['A', 'B']
l = [(x, y) for x in [1, 2] for y in ("A", "B")]
# result: [(1, 'A'), (1, 'B'), (2, 'A'), (2, 'B')]

l = [c for c in "Hello, World!" if c.isupper()]   # ['H', 'W']
```

You can create tuples and dictionaries with similar syntax.

8 Dictionaries (`dict`)

Dictionaries are key-value pairs, and use curly braces (`{}`). They work just like language dictionaries, when you have a *word* (key) you want the *definition* (value) for, you are able to find the definition easily. It doesn't work the other way, though: if you have the *definition of a word* (value) and are looking for the *word* (key), it is much more complicated.

8.1 Length, indexing, and modifying a dictionary

There is also a concept of length for dictionaries: it is the number of entries.

```
eg_dict = {"python": "snake", "boa": "snake"}
print(len(eg_dict))      # 2, since there are 2 key-value pairs
```

Indexing and modifying dictionaries are done with *keys* instead of numerical positions:

```
print(eg_dict["python"])        # Access the value of an existing key

# Modify the value of an existing key
eg_dict["python"] = "programming language"
print(eg_dict["python"])

# Define a new key-value pair
eg_dict["anaconda"] = "big snake"
print(eg_dict["anaconda"])
```

8.2 Membership operator (`in`)

The membership operator **`in`** checks if a key is in the dictionary. It does *not* check for values!

```
print("python" in eg_dict)      # True
print("cobra" in eg_dict)       # False
print("big snake" in eg_dict)   # False: that is a value in the dictionary
    , not a key
```

8.3 Dictionary methods

Again, there are some methods that act on the dictionary.

```
# Get all key:value pairs in the dictionary
for key, value in eg_dict.items(): print(key, value)

# Get all keys of the dictionary
for key in eg_dict.keys(): print(key)

# Get all values in the dictionary
for value in eg_dict.values(): print(value)

# Update dict with key:value pairs in new_dict
another_dict = {"viper": "poisonous snake!"}
print(eg_dict)
eg_dict.update(another_dict)
print(eg_dict)

print(another_dict)
another_dict.clear()            # Empty the dictionary
print(another_dict)
```

9 Iteration

You can iterate over strings, lists, tuples (below), and dictionaries (above).

```
for char in "this is a string":
  print(char, end="")

for e in ["this", "is", "a", "list"]:   # same for tuples
  print(e, end=" ")
```